

1
2
3
4
5
6
7
8
9
10
11
12
13
14

// how you ask, and what the model can see

Prompting and Context Engineering

Two prompts, same task

PROMPT · A

write a function to parse a date string

PROMPT · B

Write a Python function `parse_date(s: str) -> date`.
Accept "2026-08-12" and "12/08/2026" (day first). Raise
`ValueError` on anything else. No external libraries.

- › Which gets the better answer, and **why**?

The one idea

A prompt is not a request. It is a **specification**. Everything you leave unspecified, the model decides for you: silently, and from whatever was most common in its training data.

- › "Better prompt" does not mean "longer prompt"
 - Length is not the variable. Two things are:
 - **Decisions made:** audience, format, length, tone, what to leave out
 - **Information supplied:** facts, data and examples the model cannot know
 - Padding ("be thorough, detailed, high quality") adds words but no decisions, so the model still fills every gap with its defaults

Agenda

// part 1: how you ask

- › Tokens, the context window, and the SPEC checklist
- › Personas, step-by-step, examples: what each really does
- › Why what the model can see now matters more than wording

// part 2: what the model can see

- › How the model fills in what you didn't say
- › Debugging a failing test, with and without the code
- › Why pasting more makes answers worse, and when to restart
- › From ChatGPT to agents: where prompting is now

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Tokens, the context window, and the SPEC checklist

// part 1 • section 1 of 3

What the model actually reads, and a checklist for prompts that leave nothing to guess

What is prompting?

Prompting is writing the **instructions and information** you give an AI model so that it produces the result you want.

- › **The prompt** is everything you send: your request, plus any files, errors or examples you paste in
- › The model answers from its training and from what is in front of it: your prompt, plus anything its tools add, such as **web search results** or files it read
- › A good prompt answers the questions the model would otherwise **guess**: what to make, for whom, in what format, and what to leave alone

Tokens: the unit the model actually reads

- › A token is a chunk of text: a word, part of a word, or punctuation

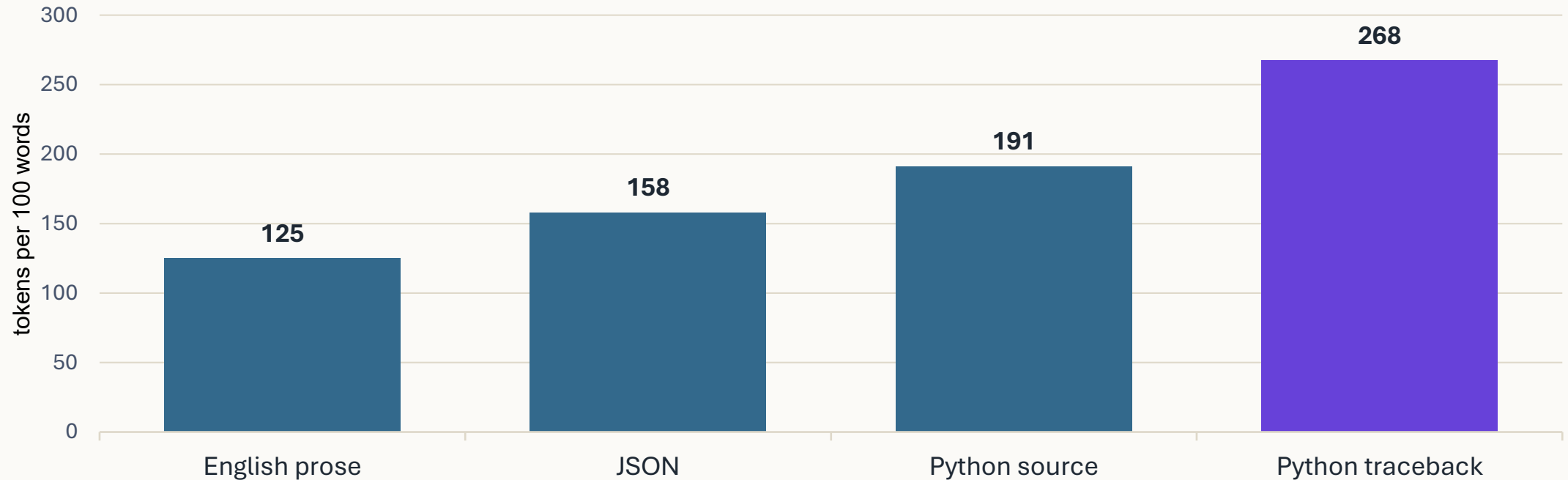
def · calculate _m edian (numbers): 6 tokens

12 / 08 / 202 6 6 tokens

Key Error : · ' unit _price ' 7 tokens

- › Cost, the size limit, and what the model can remember are all counted in tokens, not words

Pasted code and errors use more tokens than English of the same length



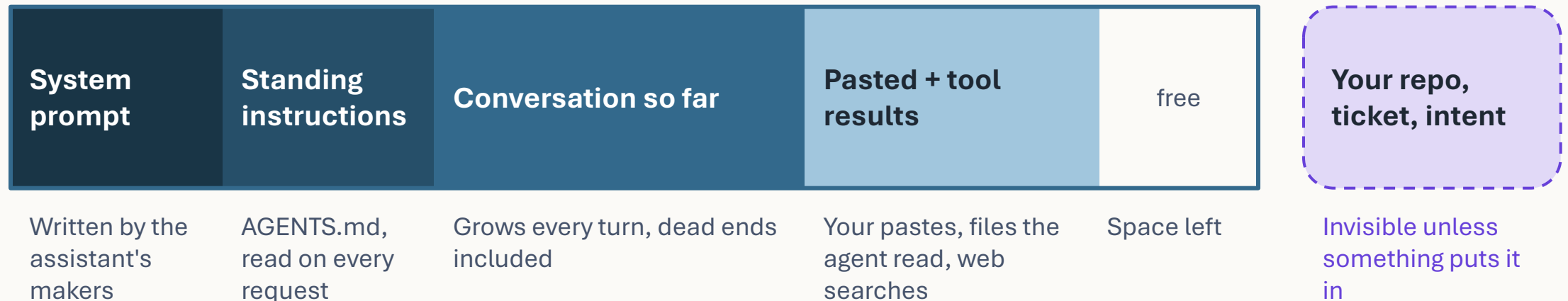
// source: counted with OpenAI's o200k_base tokenizer on samples of each kind

- › Per word, code uses more tokens than English: **1.5× for Python, 2× for a traceback**
- › Names, file paths and symbols each split into several tokens

// OpenAI's rule of thumb: 1 token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of an English word

The context window

// the context window: everything the model sees on one request



- › A chat or coding assistant is a program that fills this window and sends it to the model
- › The model sees only what is inside the bar. Anything outside it, like your repo or what you meant, does not exist for the model

SPEC, a recipe for every prompt

Letter		Example
S	Specific goal	"Parse a date string into a date"
P	Programming language / file	"Python 3.12, in utils/dates.py"
E	Example, input to output	"2026-08-12" → date(2026, 8, 12)
C	Constraints	"No external libraries. Raise on bad input"

// any checklist works; what matters is covering all four, every time

The opening pair, decoded

PROMPT · B

Write a Python function `parse_date(s: str) -> date`.
Accept "2026-08-12" and "12/08/2026" (day first). Raise
`ValueError` on anything else. No external libraries.

- › **S** parse a date · **P** Python, typed signature · **E** two example formats · **C** raise an error, no libraries

Is 12/08/2026 in August or December? Prompt A leaves the model to guess. **B is better because it makes that decision**, not because it is longer.

// tested Sept 2026: given prompt A, three current models read it as 8 December (month first), and none asked

Constraints and non-goals

Constraints: what the model must respect

- › Python 3.12
- › no new dependencies
- › under 200ms
- › no eval()
- › Assistants often **change more than you asked**. Bugs hide in those extra changes, because nobody expected them, so nobody checked them

Non-goals: what the model must not touch

- › No database schema changes
- › No error handling yet
- › Do not reformat the rest of the file

Predict: what comes back?

PROMPT

here's my file, fix the bug in calculate_total

1

Just the one-line fix

2

The fix, plus changes you did not ask for

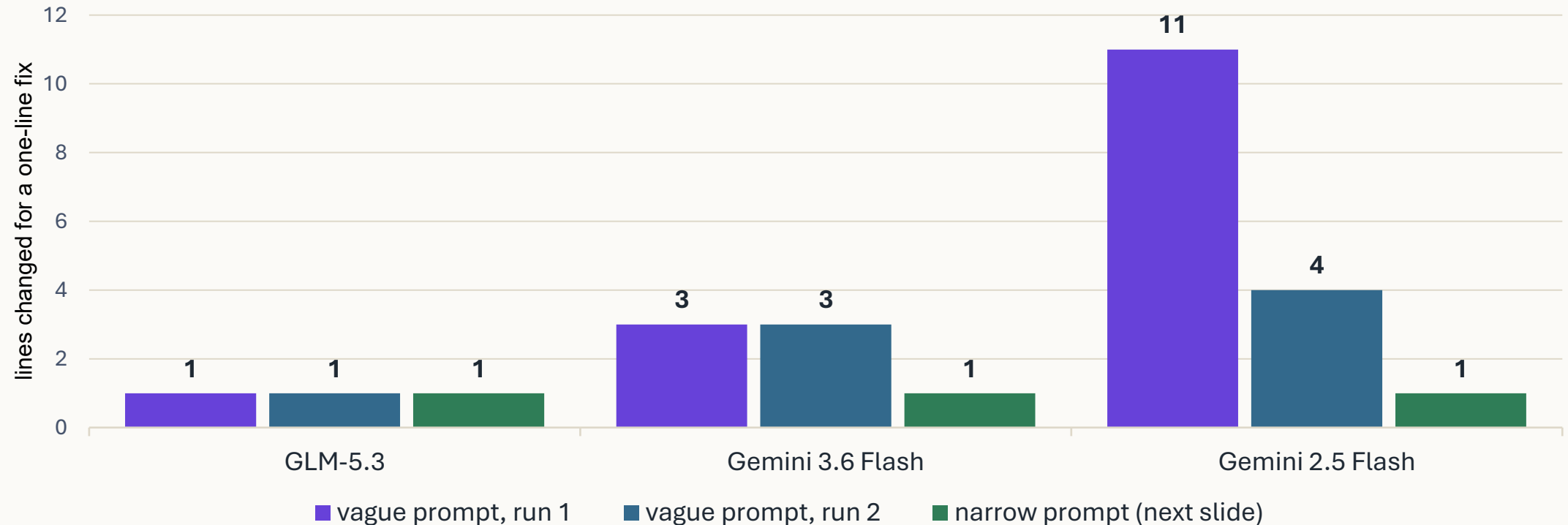
3

The whole file, reformatted, with type hints and error handling

Answer: none of them reliably. It depends on the model, and on the run

Tested Sept 2026: the same prompt changed 1 line with one model, 3 with another, and 11 then 4 with a third

A vague prompt changed 1 to 11 lines; a narrow one changed 1 every time



// source: tested Sept 2026 through each vendor's API, no system prompt; every reply fixed the bug

- › None added type hints or error handling. The extras were a rewritten loop, new comments, or the whole file sent back

Ask narrowly, and say what not to touch

PROMPT

Fix only the off-by-one in `calculate_total`.
Change nothing else: no formatting, no type hints, no error handling.
Show me a diff, not the file.

You will **read** a one-line change carefully, but only **skim** a whole file. Asking for just the fix, as a diff, is what makes a real review possible.

// tested Sept 2026: a one-line diff from all three models

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Personas, step-by-step, examples: what each really does

// part 1 • section 2 of 3

Three well-known techniques, what the evidence says about each, and when to use which

Three techniques worth knowing

Technique	What it changes	Reach for it when
Persona "You are a security reviewer"	Tone, audience, what the model focuses on	You want a lens: security, performance, a beginner
Chain-of-thought "Think step by step first"	Shows the steps, not just the answer	You need to check the reasoning, not only the result
Few-shot Two examples, then yours	A “ shot ” is one worked example: an input and its exact output. Give 2 or 3	The output format must be exact

- › Add-ons to a SPEC prompt: few-shot is SPEC's **E**, done properly
- › Each is **one prompt**, not a slow conversation or a series of prompts

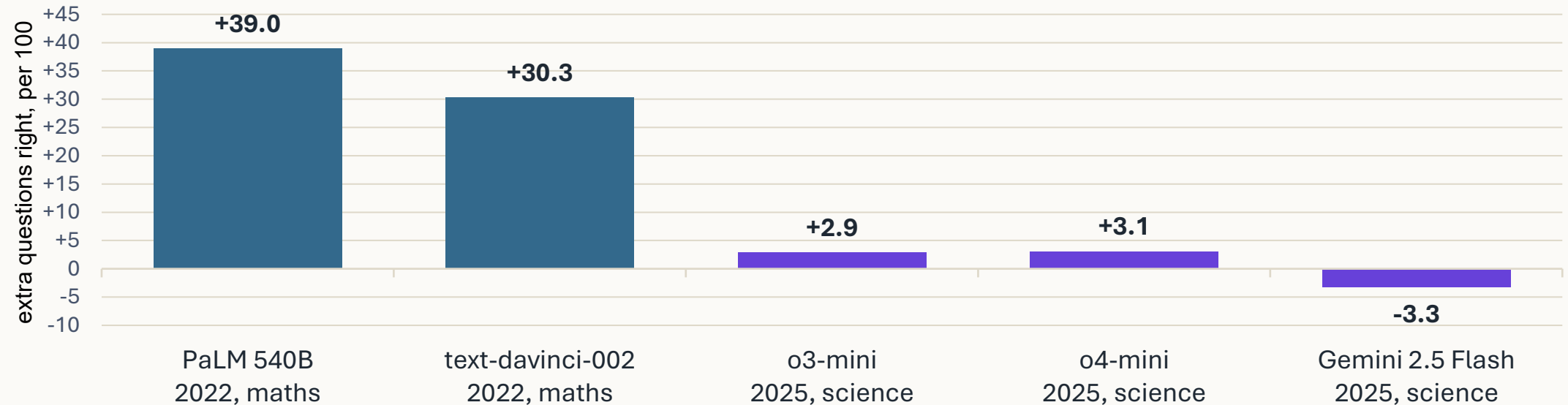
// each technique fixes one kind of problem; use it only for that problem

Personas set tone, not accuracy

- › "You are an expert" did not make answers more accurate: **162 roles**, 2,410 questions, nine open models (Zheng et al., 2024)
- › Re-tested in 2025 on GPT-4o, o3-mini, o4-mini and Gemini: no significant gain, and "layperson" personas made answers **worse**
- › What a role does change: tone, audience, and what the model focuses on

Use a persona to set the **viewpoint** ("review this as a security auditor"), not to make answers more correct

Chain-of-thought: a 2022 trick, now built in



// source: Wei et al. 2022 and Kojima et al. 2022 (GSM8K); Meincke et al., Wharton, 2025 (GPQA Diamond)

- › Today's reasoning models already work through the problem before answering, so "think step by step" barely helps and can make results worse
- › Still useful: ask the model to show its plan or its **assumptions**, so you can check them

Predict: which technique?

You need a report as CSV: fixed header, fixed column order, no commentary.
The model keeps adding a sentence of explanation above the data.

- › Persona: "you are a data engineer"
- › Chain-of-thought: "think step by step about the format"
- › Few-shot: two complete examples of the exact output
- › A longer, firmer description: "no commentary, I mean it"

Few-shot, shown

PROMPT

One CSV line per bug report. Header: severity,component,summary.

"Login 500 when the password has an emoji" → high,auth,500 on emoji password

"Typo on the about page, and in the footer" → low,web,"typo on about page, footer"

Now: "Checkout total wrong when a coupon is applied twice"

REPLY

high,checkout,coupon applied twice miscounts total

- › The two examples differ on the tricky case: the second has a comma inside a value, and shows it wrapped in quotes
- › A persona or "think step by step" would not fix this: neither shows the exact **format**. Examples do

// if code will read the output, use the API's structured-output option (a JSON schema) instead

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Why what the model can see now matters more than wording

// part 1 • section 3 of 3

When rewording stops helping, the model is usually missing information

Wording matters less than it did. Missing information matters most

THEN • 2023

Tiny wording changes swung the results

Swapping a colon, a newline or capital letters moved one model's score by up to **76 points out of 100**

So people hunted for magic phrasings

NOW

Models understand sloppy requests

Rewording the same request **changes little**

Polishing the wording is no longer where the gains are

What no wording can do: **give the model a fact it was never shown**

e.g. no rewording of “fix it” tells the model your data uses the key price, not unit_price

So when an answer is wrong, first ask: **what does the model not know?**

Context engineering

Context engineering is **choosing what the model gets to see**: which files, errors, examples and past messages go into its window, and which stay out.

	Prompt engineering	Context engineering
You ask	How do I phrase this?	What does the model need to see?
You work on	Wording, structure, examples	Files, errors, examples, history
Fails when	The request is ambiguous	The model lacks a fact it can't guess
Covers	One message	Everything in the window, every turn

- › Prompt engineering is part of it: your request is **one item** in the window
- › The goal is not more context but the **right** context: only what this task needs

// Anthropic: "the smallest possible set of high-signal tokens" for the task

Predict: three rewrites, still wrong. Why?

SCENARIO

You asked the model to fix a bug, then rewrote the request three times, each clearer than the last. The answer is still wrong in the same way.

1

The model is not good enough: switch to a bigger one

2

Keep rewording until you find the magic phrasing

3

The model is missing a fact it cannot guess

4

Turn up the randomness for a different answer

Answer: 3. The model is missing a fact it cannot guess

If clearer wording changes nothing, wording was not the problem. Rewording adds no new facts, and a bigger model or more randomness has the same gap.

Fix: give the model what it is missing, e.g. the file, the error message or the failing test.

Wrong answer? First ask which kind of problem

Wrong because I **asked** badly, or because the model doesn't know something?

You asked badly

HOW YOU CAN TELL

- The model misread what you wanted
- Each rewrite gives a different answer

WHAT TO DO

Rewrite the prompt: fill the SPEC gaps

The model lacks a fact

HOW YOU CAN TELL

- The same wrong answer every time
- The model guesses names, keys or versions

WHAT TO DO

Paste in the error, file, schema or failing test

Rule of thumb: if you can name the fact the model is missing, paste it in. If not, it's the wording.

Ask the model to ask you questions before it writes code

PROMPT

Before writing any code, ask me up to five questions about anything you would otherwise have to guess. Then stop and wait for my answers.

- › Add the line above to your prompt. The model replies with **questions, not code**
- › Each question is a decision your prompt left out, e.g. "day first or month first?"
- › You answer them, so **you** make those decisions instead of the model guessing

// then send your answers back: together with the request, they are a complete SPEC

What prompting cannot fix

- › The model cannot see your code, tickets or intent unless you, or its tools, put them in front of it
- › The model has a training cutoff: web search can fill the gap, but only if it searches, and it may find old docs
- › Exact counting and arithmetic are unreliable, **unless the model runs code** for them
- › Ask the wrong question and the model will answer it confidently, without telling you it was the wrong question

// a good prompt directs what the model can already do; it cannot add knowledge or ability

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Up next: part 2

Part 2 answers: when you leave a gap, *how* does it get filled, and what, exactly, is in front of the model when it answers?

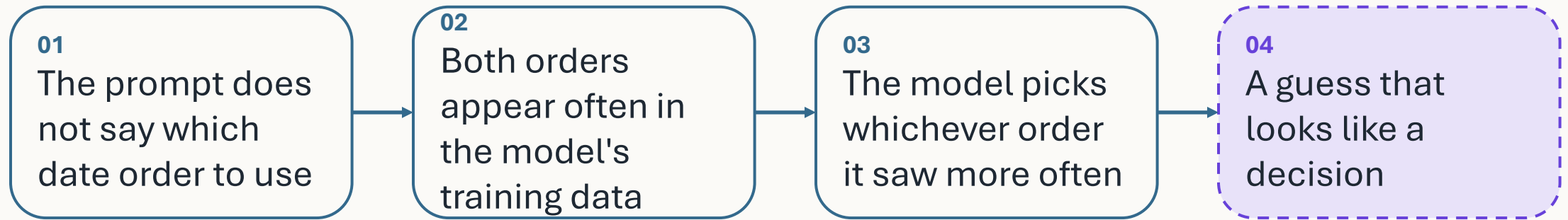
1
2
3
4
5
6
7
8
9
10
11
12
13
14

How the model fills in what you didn't say

// part 2 • section 1 of 4

Where the model's guesses come from, and what is actually in the window when it answers

How a gap gets filled



- › The model predicts the next token from **everything in the window**, and nothing else
- › Where your prompt says nothing, the model picks the most common choice from its training data, not the right one for you

So a vague prompt's guesses always look reasonable, which makes the wrong ones hard to spot.

Context you write once

```
# AGENTS.md
- Python 3.12; run the tests with `pytest -q`
- Dates are day first: 12/08/2026 is 12 August
- Money is Decimal, never float
- Never add a dependency without asking
```

- › Most coding assistants add one of these files to **every** request: AGENTS.md, CLAUDE.md, .github/copilot-instructions.md
- › Keep it short and up to date: the model follows an outdated rule just as strictly as a correct one

// anything you keep typing into every prompt belongs in this file

Predict: what does the assistant see?

Your editor's assistant. A project of forty files. You ask about a function in a file you have **not** opened.

- › The whole project: it is open in the editor
- › Everything you have opened since you started the editor
- › The file you are in, plus whatever the assistant decided to read first
- › Nothing but your question

Only what was put in the window

- › The model sees the file you are in, any text you selected, and whatever files the assistant chose to read
- › An open project is not visible to the model. It sees only what was put in the window
- › Some assistants search your files first and pick the context for you
- › When an answer is wrong, first check which files the assistant actually read

`// if the model "doesn't know" something, that thing was not in the window`

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Debugging a failing test, with and without the code

// part 2 • section 2 of 4

The same error twice: first on its own, then with the code it came from

A second case: the failing test

- › The opening pair was **generation**: write me a function. This is **diagnosis**: why is this broken?

```
FAILED tests/test_orders.py::test_total - KeyError: 'unit_price'
```

PROMPT

my test fails with KeyError: 'unit_price', fix it

- › Before the next slide: is this a wording problem, or a knowledge problem?

Predict: error only, what comes back?

- › The model asks to see the function
- › A `.get()` with a default value, so the missing key stops raising
- › Both, in the same reply
- › The model rewrites the test to match

What came back

REPLY · Gemini 2.5 Flash, Sept 2026, excerpt

Use dict.get() with a default value (Recommended). This is the most Pythonic and safest way to access dictionary keys that might not exist.

- › 3 of 4 replies asked to see the code, but all 4 still suggested .get() with a default of 0
- › Apply that fix and the test still fails, with a different error:

```
assert 0 == 10.0
```

- › The fix hid the error instead of fixing the bug

What the model could not see

```
def order_total(order: dict) -> float:  
    return sum(item["unit_price"] * item["qty"] for item in order["items"])
```

```
def test_total():  
    order = {"items": [{"price": 2.50, "qty": 4}]}  
    assert order_total(order) == 10.0
```

- › The test data uses the key `price`, but the function expects `unit_price`. Each is fine alone; they just don't match
- › No rewording of "fix it" could find this: the model was never shown this code
- › The real fix is a choice only you can make: which key name should the whole codebase use?

What to put in front of the model

Put in

Leave out

The error, **verbatim**: never your paraphrase of it

The rest of the file

The function that raised

Files that are not on the path to the error

The input that triggered it: the fixture, the request

Your guess at the cause: give the model the evidence and let it find the cause

The test, the schema, the config it depends on

The forty-minute conversation you had about it

// paste the real code and error, not a summary; include everything relevant and nothing else

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Why pasting more makes answers worse, and when to restart

// part 2 • section 3 of 4

Leaving things out on purpose, and starting fresh when a conversation goes in circles

More is not better: why

Three files pasted "just in case"

noise

Two attempts already abandoned in this conversation

noise

The four lines that actually decide the answer

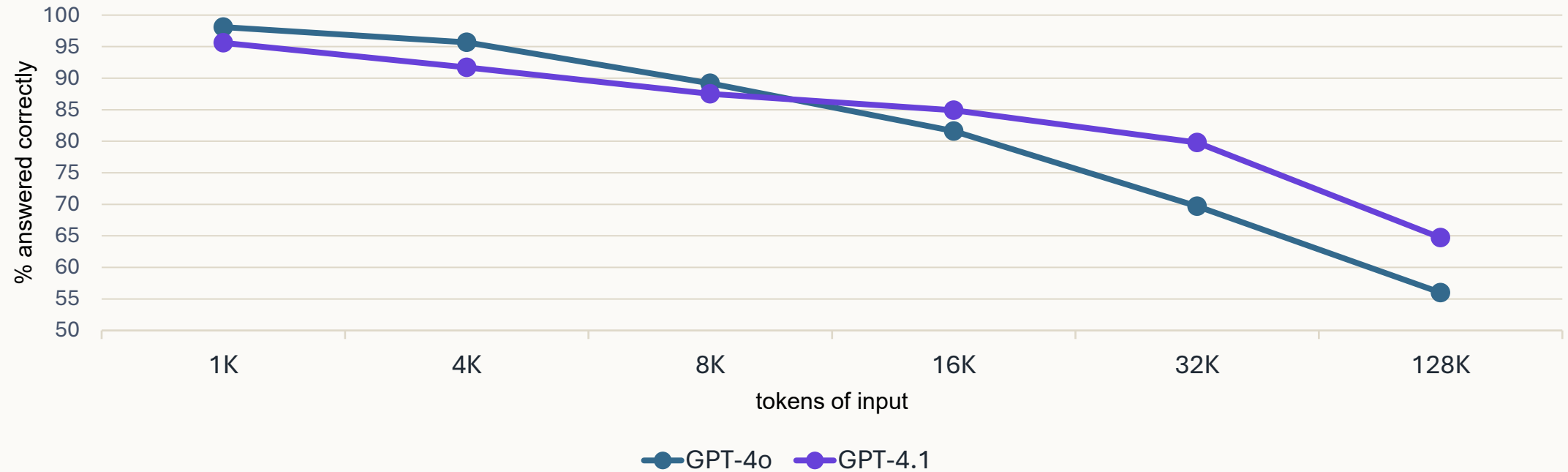
signal

A 300-line log, of which one line matters

noise

- › The model must find the few lines that matter among everything you pasted; the more you paste, the more often it misses them
- › Deciding what to leave out matters as much as deciding what to put in

Longer input, less reliable answers



// source: NoLiMa, Adobe Research, ICML 2025 (GPT-4.1 from its repository)

- › In Chroma's 2025 tests, every model answered better from a short ~300-token prompt holding just the relevant part than from the full ~113K-token conversation

// in 2023 models often missed facts in the middle of long input; that is less true now, but longer input still gives worse answers

The thread is context too

- › Every turn stays in the window: your questions, its answers, and every attempt you rejected
- › The model cannot tell which ideas you have given up on. A rejected attempt still influences it as if it were current
- › Correct the model, and the wrong version is still there, beside the correction
- › When the window is full, assistants replace the conversation with a summary, and anything the summary leaves out is lost

// a long conversation slowly fills up with old attempts you no longer want the model to use

Predict: the thread is going in circles

Forty minutes in, and each answer is a variation on the last one.

- › Push on: it is nearly there
- › Switch to a bigger model
- › New conversation, carrying only the context that turned out to matter
- › Raise the temperature for a fresh angle

Start again, carry the good context

PROMPT

New conversation. Here are `order_total`, its test, and the fixture it uses.

A default value was already tried and is wrong: the test then fails as `0 == 10.0`.

Propose the minimal fix, and state which key name you treat as canonical.

- › What you carry: the code, the failing test, and what was **ruled out**
- › What you leave behind: forty minutes of failed attempts that were still influencing every answer
- › The last line forces the model to say which key name it chose, instead of choosing silently

The best context is executable

```
def test_parse_date_day_first():  
    assert parse_date("12/08/2026") == date(2026, 8, 12)
```

- › These two lines give an **example**, a rule, and a fact the model could not know: dates here are day first
- › Prose can be read two ways. A test can only pass or fail
- › Write the test first, then ask for code that passes it. You stop judging code by how it looks and check whether it passes

// a test written after the code only checks what the code already does, bugs included

Three ways an answer goes wrong

The gap	What it looks like	The fix
Under-specified	Code that looks fine but is full of choices you never made	Make the decision yourself: add one line to the prompt
Under-informed	Sounds right but is wrong for your code; rewording does not help	Put the file, the error, the test in the window
Over-loaded	Generic, vague, or fixated on the wrong part	Remove what is not needed

Asking "bad wording or missing information?" tells the first two apart. The third, pasting too much, is rarely suspected because it feels like being thorough.

Common mistakes

When you ask

- › Adding adjectives instead of **decisions**: "robust" tells the model nothing specific
- › Using a technique (persona, step by step, examples) without a problem it fixes
- › Accepting a large change because the fix is somewhere inside it
- › Telling the model "you are an expert" to get more accurate answers

When you give the model context

- › Rewording when the real problem is missing information
- › Pasting far more than the question needs
- › Paraphrasing the error instead of pasting it verbatim
- › Assuming the model can see the project because it is open
- › Staying in a long conversation full of failed attempts

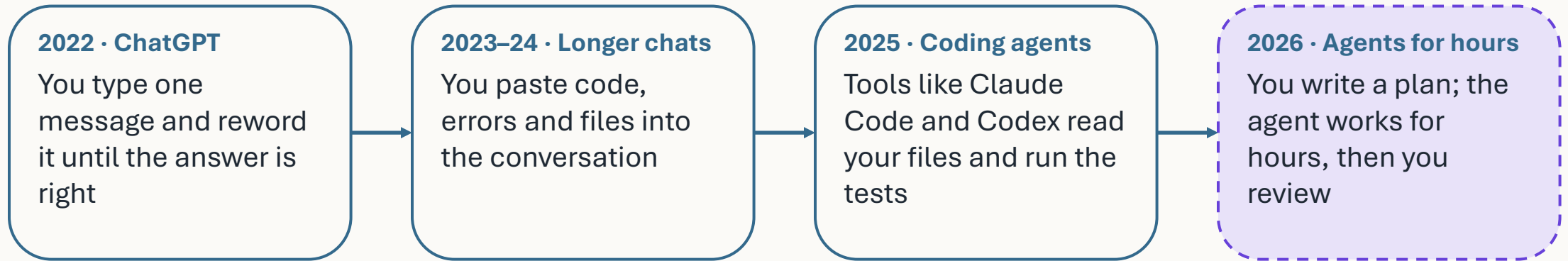
1
2
3
4
5
6
7
8
9
10
11
12
13
14

From ChatGPT to agents: where prompting is now

// part 2 • section 4 of 4

How prompting changed since 2022, and why it still matters when agents work for hours

From one message to a plan that runs for hours



- › The longest task an AI agent can finish alone grew from **under a minute** (GPT-3.5) to **about 12 hours** (early 2026)

// source: METR, task length agents complete half the time, measured in human working time, 2025–2026

Your job moved from **choosing the right words** to **deciding what gets built, and checking it**

We still prompt: the plan is now the prompt

An agent that works for hours still starts from a prompt. It is just longer: a **written plan**, plus the **standing instructions** it reads every time.

- › A vague plan fails the same way as a vague message, only after **hours of work** instead of seconds
- › The rules in this lecture apply to both: make the decisions, supply the information, say what not to touch
- › What changed is where your effort goes: **less rewording**, more writing the plan and **reviewing the result**

Summary

How you ask

- › A prompt is a **specification**: anything you leave out, the model fills with the most common choice
- › Use **SPEC** every time; saying what not to touch keeps changes small enough to review
- › Examples fix the format; personas set tone, not accuracy

What the model can see

- › The model sees only what is in the window, so check what was put there
- › Pasting more makes answers worse: leave out extras, and restart looping conversations
- › A test is the best context: it gives an example, a rule and new information at once

Is it wrong because I asked badly, or because the model doesn't know something?
No amount of rewording adds information.